

Reviewer Pack — Minimal Local Crossed Module Rank and Tseitin Formula Resolu...

Entry f394dabd54cb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 02:31:34 UTC

Minimal Local Crossed Module Rank and Tseitin Formula Resolution Proof Length

Entry ID: f394dabd54cb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 02:31:34 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology: Local Systems **Field B** (complexity object): Complexity Theory: Tseitin Formula Resolution Proof Complexity

Statement:

{‘sentence_1’: ‘For any Tseitin formula, the minimal local crossed module rank of its associated cover is polynomially related to the length of the shortest resolution proof.’, ‘sentence_2’: ‘Specifically, there exists a constant c such that for all Tseitin formulas, if they require a resolution proof of length at least n , then their minimal local crossed module rank is at least cn .’, ‘sentence_3’: ‘This polynomial relationship holds as an upper bound, meaning the local crossed module rank grows no faster than polynomially with the resolution proof length.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Local systems in algebraic topology provide a framework for understanding the structure of spaces and may reveal hidden complexity-theoretic properties.’, ‘sentence_2’: ‘The association between local crossed modules and coverings is well-studied, suggesting potential links to proof length in logic and complexity theory.’, ‘sentence_3’: ‘This conjecture aims to exploit this connection to potentially provide a new insight into the hardness of Tseitin formulas.’}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9447366ebd0b77c1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Tseitin formulas, there exists a constant c such that the minimal local crossed module rank (rank) is at most c times the resolution proof length (length), and no seed produces a ratio greater than $c+1$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'local crossed module rank' AND 'Tseitin formula' AND resolution proof
- 'polynomial relationship' AND 'minimal local crossed module rank' AND Tseitin formula -
resolution proof length AND 'algebraic topology local systems' AND complexity theory

Top relevant hits considered: - [s2:10.4171/owr/2014/52] Mathematical Logic: Proof Theory,
Constructive Mathematics - [s2:10.1088/0264-9381/13/12/022] Complex Systems and Binary Net-
works

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1,
elapsed=0.1s

5.1 Generated Python source

```
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
```

```

    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def tseitin_formula(n):
    variables = [f'x{i}' for i in range(1, n+1)]
    clauses = []
    for i in range(1, n+1):
        clauses.append([variables[i-1]])
    for i in range(1, n+1):
        for j in range(i+1, n+1):
            clauses.append([-variables[i-1], -variables[j-1]])
            clauses.append([variables[i-1], variables[j-1]])
    return variables, clauses

def local_crossed_module_rank(clauses):
    m = len(clauses)
    A = [[0] * (m + 1) for _ in range(m)]
    for i in range(m):
        for j in range(i+1, m):
            if any(c in clauses[i] and -c in clauses[j] for c in clauses[i]):
                A[i][j] = 1
                A[j][i] = 1

    rank = 0
    for i in range(m):
        if all(A[j][i] == 0 for j in range(i)):
            rank += 1
    return rank

def resolution_proof_length(clauses):
    stack = []
    while clauses:
        clause = clauses.pop()
        if len(clause) == 1:
            literal = clause[0]
            if literal > 0 and -literal in [c for cl in clauses for c in cl]:
                continue
            elif literal < 0 and -literal not in [c for cl in clauses for c in cl]:
                return float('inf')
        else:
            literal = random.choice(clause)
            if literal > 0:
                stack.append(-literal)
            else:
                stack.append(literal)
        new_clauses = []
        for cl in clauses:
            if literal not in cl and -literal not in cl:
                new_clauses.append(cl)
            elif literal in cl:
                new_clauses.extend([c for c in cl if c != literal] + [-l for l in stack])
            else:
                new_clauses.extend([c for c in cl if c != -literal] + [l for l in stack])
        clauses = new_clauses

```

```

    return len(stack)

n = random.randint(5, 40)
variables, clauses = tseitin_formula(n)
rank = local_crossed_module_rank(clauses)
length = resolution_proof_length(clauses)

if length == float('inf'):
    return {
        "metric_name": "local_crossed_module_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "resolution_proved_unsat"
    }

c = rank / length
if c > 1:
    return {
        "metric_name": "local_crossed_module_rank",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"c={c} exceeds 1"
    }

return {
    "metric_name": "local_crossed_module_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]), "")
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5adb24aa.py", line 138, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5adb24aa.py", line 101, in run_trial
    variables, clauses = tseitin_formula(n)
                        ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5adb24aa.py", line 55, in tseitin_formula
    clauses.append([-variables[i-1], -variables[j-1]])
                  ~~~~~
```

TypeError: bad operand type for unary -: 'str'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to complete the required computations to verify the conjecture. | next: Review and debug the test code to ensure it can run to completion without errors.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15599
2	preregistration	ollama_remote	glm4:latest	0	0	9085
3	novelty	ollama_remote	glm4:latest	0	0	8403
4	novelty	ollama_remote	glm4:latest	0	0	9117
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40886
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9180
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16877
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16208
9	judge	ollama_remote	glm4:latest	0	0	14707

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140061 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f394dabd54cb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f394dabd54cb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f394dabd54cb.tar.gz` (if generated)